

SCHOOL OS SaaS

Project Management Handbook

Execution baseline for planning, delivery, quality, release and improvement

Version	0.1
Baseline date	19 September 2026
Project start	21 September 2026
Target GA	17 April 2028
Companion specification	School_OS_SaaS_Master_Specification_v0.1

Living document - improve by versioning, not by silently replacing decisions.

Contents

- 1. Purpose and project context
- 2. Delivery assumptions
- 3. Scope map
- 4. Product and architecture principles
- 5. Master timeline
- 6. Release strategy and milestones
- 7. Delivery cadence and work rules
- 8. Governance, RACI and change control
- 9. Quality, security and data management
- 10. SaaS deployment and environment strategy
- 11. Testing and defect management
- 12. Risk management
- 13. Documentation, reporting and KPIs
- 14. Pilot, UAT, release and operations
- 15. Improvement backlog
- Appendices - management checklists and package files

1. Purpose and project context

This handbook is the execution companion to the School OS SaaS Master Specification. It defines how the product is planned, sequenced, built, tested, reviewed, released, supported and improved. It is not the database schema; it is the management system that controls the work leading to that schema and product.

The product is a configurable School Operating System built around five cross-cutting engines: Data, Permission, Workflow, Automation and AI. The management approach protects those principles while allowing iterative releases.

2. Delivery assumptions

- One customer school uses one Supabase project for strong tenant isolation.
- A separate SaaS control plane tracks customers/projects/subscriptions/versions and does not hold school operational data.
- Flutter Windows (.exe) is the first desktop client.
- Supabase/PostgreSQL provides database, Auth, Storage, RLS and server-side functions/services.
- Two-week sprints are used, with architecture and release gates.
- Baseline dates assume one lead engineer with part-time QA/design/domain support.
- English is initial language; localization is kept possible.
- Generated PDFs are created on demand rather than auto-stored.
- Financial transactions are reversed/cancelled, never casually hard-deleted.
- GPS, biometrics, inventory, WhatsApp and external payment gateway are deferred.

3. Scope map

Area	Included capability
Foundation	SaaS control plane, authentication, people/accounts, RBAC, scopes, school config, campuses, academic years, classes/sections/rooms
Student Core	Students, families, admissions, enrollment, transfers, promotion, student status/history, documents
Academic	Curriculum, subjects, teachers, teacher assignments, timetable, attendance, homework, assessments, marks, results
Finance	Fee types/structures, discounts, invoice data, payments, verification, receipts, advance/partial payments, credit/refund, reversals, reports
Administration	HR, employee attendance, leave, payroll, contracts, documents and approvals
Student Life	Library, transport, hostel and medical
Platform	Approval engine, audit, notifications, search, analytics, automation, AI, offline sync, backup/recovery
Deferred	GPS, biometrics, inventory, WhatsApp and external payment gateway

4. Product and architecture principles

1. History over overwrite: preserve academic, financial, result, status and audit history.
2. Permission is not authority: users may be allowed to request a change without applying it directly.
3. Approval is reusable: sensitive changes use a central workflow/approval model.
4. Audit is mandatory: sensitive actions, including Super Admin actions, are traceable.

5. Default-deny security: access is explicitly granted through role, action and scope.
6. AI obeys the same permission boundaries as the normal application.
7. Configuration beats customer code forks.
8. Database changes are controlled, versioned migrations.
9. Each school project must be reproducible from deployment documentation.
10. Offline sync is conflict-aware and does not silently overwrite critical records.

5. Master timeline

Phase	Name	Start	End	Gate
P0	Discovery & Project Setup	2026-09-21	2026-10-16	Requirements Baseline
P1	Architecture & SaaS Foundation	2026-10-19	2026-12-11	Architecture Gate
P2	Identity, RBAC & Academic Foundation	2026-12-14	2027-02-12	Foundation Alpha
P3	Students, Families & Admissions	2027-02-15	2027-04-09	Student Core Alpha
P4	Curriculum, Teachers & Timetable	2027-03-22	2027-05-14	Academic Alpha
P5	Attendance, Homework & Notifications Core	2027-04-26	2027-06-25	Operations Alpha
P6	Exams, Marks & Results	2027-06-14	2027-08-20	Academic Pilot
P7	Fees, Payments & Finance	2027-08-09	2027-10-22	School Beta
P8	HR, Payroll & Leave	2027-10-04	2027-11-26	Admin Beta
P9	Library, Transport, Hostel & Medical	2027-11-08	2028-01-14	Feature Complete
P10	Offline Sync, Backup & Hardening	2027-12-13	2028-02-11	Resilience Gate
P11	Analytics, Automation & AI	2028-01-17	2028-03-17	Intelligence RC
P12	UAT, Security, Performance & Launch	2028-03-01	2028-04-14	GA Readiness

The detailed workbook breaks this roadmap into 108 tasks and 42 two-week sprints with owners, dependencies, priorities, acceptance criteria, progress tracking and release mapping.

6. Release strategy and milestones

6.1 Release sequence

Release	Name	Scope
R0	Internal Foundation	Identity/RBAC/campus/academic foundations.
R1	Student Alpha	Students, family, admissions and enrollment.
R2	Operations Alpha	Timetable, attendance, homework and notifications.
R3	Academic Pilot	Exams, marks and results.
R4	School Beta	Fees, payments and financial controls.
R5	Feature Complete	HR and optional operational modules.
R6	Release Candidate	Offline, backup, analytics, automation and AI.
R7	General Availability	Production release after all blocking gates.

6.2 Major milestones

ID	Milestone	Target	Exit intent
M01	Requirements Baseline	2026-10-16	Requirements and PM baseline approved.
M02	Architecture Gate	2026-12-11	SaaS, DB, RLS, storage, migrations and deployment patterns approved.
M03	Foundation Alpha	2027-02-12	Authentication, RBAC, scope, campus and academic foundation working.
M04	Student Core Alpha	2027-04-09	Admissions, family, enrollment and promotion history working.
M05	Operations Alpha	2027-06-25	Timetable, attendance, homework and base notifications usable.
M06	Academic Pilot	2027-08-20	Exam, marks, results and result card lifecycle usable.
M07	School Beta	2027-10-22	Finance/payments integrated with academic core.
M08	Feature Complete	2028-01-14	Core optional modules completed.
M09	Resilience Gate	2028-02-11	Offline sync, backup/restore and deployment rehearsal pass.
M10	Intelligence RC	2028-03-17	Analytics, automation and permission-aware AI complete.
M11	GA Readiness	2028-04-14	UAT, security, performance and documentation gates pass.
M12	General Availability	2028-04-17	Production release.

7. Delivery cadence and work rules

7.1 Cadence

- Sprint length: 2 weeks.
- Sprint planning: select dependency-ready work under a single sprint goal.
- Weekly risk/decision/security/migration review.
- Sprint review/demo against acceptance criteria.
- Sprint retrospective and improvement actions.
- Phase gate before treating foundations as stable.
- Release gate before external/pilot/production promotion.

7.2 Definition of Ready

A task is Ready when the objective, owner, dependencies, acceptance criteria, permission/data impact and test expectation are understood. Schema work must also describe naming/history/RLS implications.

7.3 Definition of Done

Done requires implementation, migrations where applicable, authorization/RLS, tests, audit/approval behavior where applicable, documentation and acceptance criteria. UI completion alone is not Done.

7.4 Priority rules

Priority	Meaning
Critical	Security/data-integrity/release blocker.
High	Required for planned release or core workflow.
Medium	Important but can move without invalidating core release.
Low	Optional/deferred enhancement.

8. Governance, RACI and change control

8.1 Core responsibilities

- Product Owner - scope, sequencing, value, change approval and go/no-go.
- Lead Engineer - architecture, schema, migrations, implementation, technical quality and security.
- QA - test strategy, regression, negative authorization tests, evidence and release quality.
- School SME - workflow realism and usability validation.
- Exam Controller - acceptance of result review/publish controls.
- Finance/Admin SME - validation of fees, payments, payroll and reconciliation.
- Operations/Support - onboarding, deployments, production support and incidents.

8.2 Change control

Major scope, architecture, security, financial-integrity and release decisions are added to the Change Log. Approved decisions are never silently overwritten; a later decision supersedes them with a new record. A major scope addition during an active sprint requires an explicit trade-off: remove equivalent work, move the addition, add capacity, or move the date.

9. Quality, security and data management

9.1 Quality gates

- Repeatable fresh-install and upgrade migrations.
- Positive and negative RLS/authorization tests.
- Audit tests for sensitive actions.
- Approval-workflow bypass tests.
- Attendance reconciliation including corrections and offline behavior.
- Result entry/correction/review/publish/lock lifecycle.
- Finance reconciliation across discounts, advance/partial payments and reversals.
- Private Storage access and document-size controls.
- Offline idempotency/conflict tests.
- Backup/restore rehearsal.
- AI authorization tests.
- Release rollback/recovery rehearsal.

9.2 Security model

Authorization is modeled as Role + Permission Action + Scope + Approval Authority. Client applications never contain service-role/database/SMTP/AI secrets. UI visibility is not considered a security control; server-side/RLS enforcement is mandatory.

9.3 Data integrity

- Preserve historical enrollment/status/result/financial/audit records.
- No hard deletion of official financial transactions.
- Published results require controlled correction/revision.
- Sensitive correction workflows retain old value, new value, reason, actor and approval.
- Schema changes use migrations; released migrations are not casually edited.

10. SaaS deployment and environment strategy

10.1 Tenant model

The chosen architecture is one school = one Supabase project. The shared control plane stores routing, project metadata, plans/modules and app/schema versions. Student, employee, finance, medical and other operational data remains in the school project.

10.2 Deployment sequence

1. Create Supabase project.
2. Configure environment and server-side secrets.
3. Run versioned database migrations.
4. Configure Storage buckets and access policies.
5. Deploy server functions/services.
6. Create initial Super Admin.
7. Run school setup/configuration.
8. Run smoke/security checks.
9. Record deployed schema/app version in control plane.
10. Activate school tenant.

10.3 Environments

Environment	Purpose
Local/Development	Rapid engineering using disposable data.
Staging/QA	Production-like schema, controlled test data and release validation.
Production	Customer school project; migrations only through controlled release procedure.

11. Testing and defect management

11.1 Test layers

- Unit/domain logic tests.
- Database constraints and migration tests.
- RLS/authorization tests including denied cases.
- Service/Edge Function tests.
- Flutter integration tests.
- End-to-end scenario tests.
- UAT with realistic school roles.
- Security/performance/recovery tests before GA.

11.2 High-value scenario packs

Admission -> enrollment -> promotion/transfer; teacher assignment and scope; morning/QR attendance and correction; exam marks -> correction -> Exam Controller review -> publish -> lock; discounts/advance/partial payment -> verification -> receipt -> reversal; payroll finalization; family account; medical privacy; offline conflict; backup/restore; AI authorization.

11.3 Defect severity

Severity	Definition	Release impact
S1 Critical	Security breach, data loss/corruption, finance/result integrity failure, core workflow unusable.	Blocks release.
S2 High	Major workflow broken with no acceptable workaround.	Normally blocks release.
S3 Medium	Functional defect with workaround.	Schedule according to release impact.
S4 Low	Cosmetic or minor usability issue.	Does not normally block release.

12. Risk management

ID	Risk	Score	Primary mitigation
R-01	Scope grows faster than delivery capacity	20	Use release gates; defer non-core features; require trade-offs for additions.
R-02	Premature schema decisions create expensive migrations	15	Use dependency-first design, ADRs, migration rehearsal and architecture gate.
R-03	RLS misconfiguration exposes sensitive data	10	Default-deny RLS, positive/negative authorization tests and security review.
R-04	Offline conflicts corrupt attendance or finance	15	Idempotent sync, conflict detection and administrator resolution; no silent overwrite.
R-05	Financial corrections become untraceable	10	No hard deletion; use reversal/cancellation, reason, approval and audit.
R-06	Lean team becomes delivery bottleneck	16	Strict priorities, test automation, documentation and additional QA/dev support when needed.
R-07	AI reveals unauthorized data	10	Permission-aware AI service layer; AI never bypasses

			normal authorization.
R-13	Customer customization forks codebase	15	Configuration and feature flags only; avoid customer-specific code branches.
R-14	Schema versions differ across school projects	15	Schema-version table, migration logs and controlled rollout checks.
R-15	Backups exist but restore is untested	10	Scheduled restore drills and GA blocking recovery rehearsal.

Scores 15+ require explicit weekly attention. The complete editable risk register is maintained in the Excel workbook.

13. Documentation, reporting and KPIs

13.1 Required documentation

- Master product requirements.
- Architecture decision records.
- Schema/data dictionary.
- Migration notes and schema versioning.
- RLS/permission matrix.
- Service contracts.
- Deployment and backup/restore guides.
- Admin and role-based user guides.
- Release notes and known issues.
- Support/incident runbook.

13.2 Weekly status report

Report completed work, current sprint goal, next milestone, overall completion, blockers, top risks, architecture decisions, security/data-integrity concerns, defects by severity and any scope/date changes. The Excel Dashboard is the numerical source; the narrative should explain deviations and decisions.

13.3 Execution KPIs

- Sprint goal completion rate.
- Critical/high tasks completed vs planned.
- Open blocker count and age.
- High-risk count and trend.
- Escaped defects by severity.
- Authorization/RLS test pass rate.
- Migration success rate.
- UAT scenario pass rate.
- Backup/restore rehearsal success.
- Schema-version consistency across school projects.

14. Pilot, UAT, release and operations

14.1 Pilot/UAT

Pilot coverage should include Super Admin/Owner/Principal, campus admin, teacher, Exam Controller, accountant, parent, student and optional-module roles. UAT uses realistic end-to-end school scenarios. Feedback is classified as defect, usability improvement, configuration need or new scope.

14.2 Release procedure

1. Freeze the migration/release set.
2. Back up and confirm recovery path.
3. Validate app/schema compatibility.
4. Run blocking QA/security/data-integrity tests.
5. Publish release notes.
6. Migrate in controlled order and confirm schema version.
7. Run smoke tests for critical roles/workflows.
8. Monitor and record release health.
9. If blocking failure occurs, use documented rollback/recovery; do not improvise irreversible SQL.

14.3 Post-GA operations

Operate with incident severity, controlled release channels, per-school version tracking, recurring restore drills, security review, cost/storage monitoring, performance monitoring and a managed backlog. Customer-specific requests should become configurable product features rather than private code forks.

15. Improvement backlog

Version 0.1 is a baseline. Later versions may add effort points, budget/cost control, automated CI/CD provisioning, SLA/support metrics, telemetry/observability targets, licensing/subscription automation, multilingual delivery, mobile clients and the deferred GPS/biometrics/inventory/WhatsApp/payment-gateway capabilities once the core platform is stable.

Appendix A - Weekly / Sprint / Gate Checklists

Weekly

- Update task status/progress/notes.
- Review blockers and high risks.
- Review migration/RLS/security changes.
- Confirm sprint goal remains realistic.
- Record major decisions in Change Log.

Sprint end

- Demo completed behavior.
- Confirm acceptance criteria and tests.
- Close or re-plan incomplete tasks.
- Update milestone/release confidence.
- Record retrospective improvements.

Phase gate

- Confirm blocking exit criteria.
- Confirm dependent schema/contracts are stable enough.
- Review security, migration and backup impact.
- Approve next phase or record remaining conditions.

Release

- Blocking tests pass.
- Migration and recovery plan ready.
- Release notes complete.
- UAT sign-off where required.
- No unresolved Severity 1 defects.
- Go/no-go recorded.

Appendix B - Package files

- School_OS_SaaS_Project_Management_Master_v0.1.xlsx - detailed workbook with dashboard, roadmap, WBS, sprints, releases, milestones, risks, RACI, QA gates and change log.
- School_OS_SaaS_Project_Management_Handbook_v0.1.md - editable source.
- School_OS_SaaS_Project_Management_Handbook_v0.1.docx - formatted execution handbook.
- School_OS_SaaS_Project_Management_Handbook_v0.1.pdf - share/presentation version.
- School_OS_SaaS_Master_Specification_v0.1.* - requirements and architecture baseline.